# Módulo de Autenticação de Usuários

> ## SignIn Use Case

## 1. Estratégia de especificação textual

- Para especificar textualmente um caso de uso de **signin** em uma API REST, a estratégia mais adequada é adotar a notação de **Cockburn** como base principal
- A notação de **Use Case 2.0** pode ser utilizada apenas quando houver interesse em fatiar a implementação em incrementos menores
- Essa escolha faz sentido porque o caso de uso de signin possui:
  - um objetivo bastante claro do ponto de vista do usuário
  - um fluxo principal relativamente linear
  - extensões bem delimitadas
- A notação de Cockburn é especialmente útil nesse cenário porque permite explicitar com clareza:
  - o objetivo
  - o gatilho
  - as pré-condições
  - o fluxo principal
  - as extensões
  - as garantias do caso de uso
- Já o Use Case 2.0 pode ser incorporado depois, sobretudo para:
  - organizar slices de implementação
  - organizar slices de testes

## 2. Delimitação arquitetural segundo Clean Architecture

### 2.1. Mapeamento de camadas

| Nome da camada | Equivalente no CA |
|---|---|
| Domain | Entities, regras de negócio |
| Application | Use cases |
| Infrastructure | Controller, gateways, adapters, presenters etc. |

## 2.2. Leitura arquitetural do caso de uso

- Aplicando essa separação ao caso de uso de signin:
  - a camada **Domain** concentra os conceitos do domínio relacionados à autenticação, como:
    - conta de usuário
    - identidade
    - eventuais regras de negócio associadas
  - a camada **Application** contém o caso de uso `SignIn`, responsável por orquestrar o fluxo
  - a camada **Infrastructure** contém os elementos concretos, como:
    - controller HTTP
    - implementações de gateways
    - demais adaptadores

## 2.3. Ponto arquitetural importante

- Sob a ótica da Clean Architecture, há um ponto fundamental:
  - embora as implementações concretas estejam na infraestrutura
  - as **abstrações necessárias ao caso de uso** devem ser definidas de modo que a aplicação dependa apenas de **contratos**
  - a aplicação não deve depender de detalhes concretos
- Trata-se de uma aplicação direta do **Dependency Inversion Principle (DIP)**

# 3. Leitura arquitetural do caso de uso `SignIn`

- O caso de uso `SignIn` deve ser compreendido como um **orquestrador de aplicação**
- Ele não deve conhecer:
  - detalhes de protocolo HTTP
  - códigos de status
  - banco de dados
  - mecanismo específico de geração de token
  - ORM
  - framework web
- Sua responsabilidade é coordenar o fluxo de autenticação com base em interfaces

## 3.1. Ações executadas por meio de contratos

- Nesse cenário, o serviço executa, por meio de contratos, as seguintes ações:
  i. verifica as credenciais de acesso
  ii. carrega os dados da conta do usuário
  iii. gera o token de acesso

iv. persiste o token gerado
v. devolve um resultado de sucesso ou falha

### 3.2. Relação com princípios do SOLID

- Essa modelagem preserva princípios relevantes do SOLID:
  - **SRP**
    - o caso de uso possui uma única responsabilidade, que é realizar o signin
  - **DIP**
    - a aplicação depende de abstrações
  - **ISP**
    - cada interface cumpre um papel específico
  - **OCP**
    - novas implementações podem ser introduzidas sem alteração da lógica central do caso de uso

# Especificação textual em notação Cockburn

## 4. Caso de uso

- **SignIn de usuário**

## 5. Escopo

- **API REST de autenticação**

## 6. Nível

- **Objetivo do usuário**

## 7. Ator primário

- **Usuário não autenticado**
- Em um sistema REST, também se pode dizer, com maior precisão, que o ator externo imediato é o:
  - **cliente da API em nome do usuário**

# 8. Interessados e interesses

## 8.1. Usuário

- deseja autenticar-se com credenciais válidas
- deseja obter acesso ao sistema de maneira segura

## 8.2. Sistema

- deve autenticar apenas credenciais válidas
- não deve emitir token sem autenticação prévia
- não deve considerar sucesso um fluxo em que o token não tenha sido persistido
- deve manter a regra de aplicação desacoplada dos detalhes técnicos

## 8.3. Equipe de desenvolvimento

- deseja um caso de uso coeso, testável e desacoplado
- deseja poder substituir mecanismos de autenticação, criptografia e persistência sem reescrever a regra central

# 9. Gatilho

- O usuário envia uma requisição de signin contendo:
  - `username`
  - `password`

# 10. Pré-condições

- a requisição chegou ao sistema
- a estrutura mínima da entrada já foi validada no adaptador de entrada
- o caso de uso recebeu dados suficientes para iniciar sua execução

# 11. Garantias mínimas

- credenciais inválidas não geram token
- não há retorno de sucesso sem persistência do token
- o caso de uso não depende diretamente de detalhes concretos da infraestrutura

# 12. Garantias de sucesso

- as credenciais foram verificadas com sucesso

- os dados do usuário autenticado foram carregados
- um token de acesso foi gerado
- o token foi persistido
- o resultado do processo foi devolvido ao adaptador de saída

# 13. Fluxo principal de sucesso

1. O usuário envia `username` e `password` ao endpoint de signin
2. O controller valida a estrutura da requisição e encaminha o input ao caso de uso `SignIn`
3. O caso de uso solicita à interface `UserAuth` a verificação das credenciais
4. A interface `UserAuth` confirma que as credenciais são válidas
5. O caso de uso solicita à interface `LoadUserAccount` o carregamento dos dados do usuário autenticado
6. A interface `LoadUserAccount` retorna os dados da conta
7. O caso de uso solicita à interface `TokenEncrypter` a geração do token de acesso
8. A interface `TokenEncrypter` retorna o token gerado
9. O caso de uso solicita à interface `SaveUserToken` a persistência do token associado ao usuário
10. A interface `SaveUserToken` confirma a persistência
11. O caso de uso devolve um resultado de sucesso contendo o token
12. O controller traduz esse resultado para a resposta HTTP adequada

# 14. Extensões

## 14.1. 2a. Requisição malformada

- 2a1. O controller identifica ausência ou invalidade estrutural de campos obrigatórios
- 2a2. O caso de uso não é executado
- 2a3. O sistema retorna erro de requisição inválida

## 14.2. 3a. Credenciais inválidas

- 3a1. `UserAuth` informa que as credenciais são inválidas
- 3a2. O caso de uso encerra com falha de autenticação
- 3a3. Nenhum token é gerado
- 3a4. Nenhum token é persistido
- 3a5. O sistema retorna falha de autenticação

### 14.3. 5a. Falha ao carregar a conta do usuário

- 5a1. `LoadUserAccount` não encontra ou não consegue recuperar os dados do usuário autenticado
- 5a2. O caso de uso encerra com falha técnica ou inconsistência interna
- 5a3. Nenhum token é gerado
- 5a4. Nenhum token é persistido
- 5a5. O sistema retorna erro apropriado segundo sua política de tratamento

### 14.4. 7a. Falha na geração do token

- 7a1. `TokenEncrypter` falha ao gerar o token
- 7a2. O caso de uso encerra com falha técnica
- 7a3. Nenhum token é persistido
- 7a4. O sistema retorna erro apropriado

### 14.5. 9a. Falha na persistência do token

- 9a1. `SaveUserToken` falha ao persistir o token
- 9a2. O caso de uso encerra com falha técnica
- 9a3. O fluxo não pode ser considerado bem-sucedido
- 9a4. O sistema retorna erro apropriado

## 15. Requisitos especiais

- o caso de uso não deve expor detalhes sobre qual parte da credencial falhou
- o caso de uso não deve depender de protocolo HTTP
- o caso de uso deve ser integralmente testável com test doubles
- o token só pode ser devolvido como sucesso se tiver sido persistido com sucesso
- o mecanismo de autenticação deve poder ser substituído sem alteração da lógica do caso de uso
- o mecanismo de geração de token também deve ser substituível sem alteração da lógica central

## 16. Observação importante de modelagem

- É importante distinguir duas responsabilidades:
  - a validação estrutural da entrada, como:
    - presença de campos obrigatórios
    - formato básico da requisição
  - pertence ao **adaptador de entrada**
  - isto é, ao controller ou a componentes associados

- Já a decisão de autenticar ou rejeitar credenciais:
  - pertence ao **fluxo de aplicação**
  - mediado pela interface `UserAuth`

# Contratos conceituais das interfaces

## 17. `UserAuth`

- Responsável por verificar se as credenciais informadas correspondem a uma autenticação válida

## 18. `LoadUserAccount`

- Responsável por carregar os dados da conta do usuário autenticado

## 19. `TokenEncrypter`

- Responsável por gerar um token de acesso a partir dos dados necessários do usuário autenticado

## 20. `SaveUserToken`

- Responsável por persistir o token emitido e associá-lo ao usuário correspondente

# Resultado semântico do caso de uso

## 21. Tipos de resultado produzidos pela aplicação

- Do ponto de vista da aplicação, o caso de uso pode produzir essencialmente dois tipos de resultado

### 21.1. Sucesso

- token gerado e persistido

### 21.2. Falha

- credenciais inválidas
- falha ao carregar a conta

- falha ao gerar o token
- falha ao persistir o token

### 21.3. Observação arquitetural

- Esse ponto é importante porque evita acoplamento indevido entre:
  - a camada de aplicação
  - a camada de transporte
- Em outras palavras, o caso de uso não deve retornar diretamente:
  - `200`
  - `401`
  - `500`
- Essa tradução pertence ao:
  - controller
  - e, eventualmente, ao presenter

# Complemento opcional com Use Case 2.0

## 22. Finalidade do Use Case 2.0

- Quando se deseja organizar a implementação de forma incremental, o **Use Case 2.0** pode ser usado como complemento

## 23. Use Case

- **SignIn**

## 24. Fluxo básico

- O usuário informa credenciais
- O sistema autentica
- O sistema carrega a conta
- O sistema gera o token
- O sistema persiste esse token
- O sistema devolve sucesso

## 25. Fluxos alternativos

- credenciais inválidas
- requisição inválida
- falha ao carregar a conta
- falha ao gerar o token
- falha ao persistir o token

## 26. Slices recomendadas

### 26.1. Slice 1 — Signin com sucesso

- Implementa o fluxo principal completo

### 26.2. Slice 2 — Rejeição de credenciais inválidas

- Implementa a falha de autenticação sem geração de token

### 26.3. Slice 3 — Rejeição de input inválido

- Implementa a validação estrutural no controller, antes da execução do caso de uso

### 26.4. Slice 4 — Falha segura na geração do token

- Implementa o tratamento de erro em `TokenEncrypter`

### 26.5. Slice 5 — Falha segura na persistência do token

- Implementa a garantia de que não há sucesso sem persistência do token

# Fechamento didático

## 27. Síntese final

- Para este caso, a formulação mais adequada é a seguinte:
  - usar **Cockburn como notação principal**
  - usar **Use Case 2.0 apenas como apoio**, caso se queira decompor a entrega em fatias menores
- Do ponto de vista arquitetural, o mais importante é perceber que o `SignIn`:
  - não deve ser tratado como controller

    - não deve ser tratado como serviço de infraestrutura
    - não deve ser tratado como detalhe de framework
- Ele deve ser tratado como:
    - **caso de uso da aplicação**
    - responsável por coordenar o fluxo por meio de interfaces:
        - pequenas
        - coesas
        - substituíveis
- É exatamente esse tipo de modelagem que permite alinhar o problema simultaneamente com:
    - **SOLID**
    - **Clean Architecture**